

Unsupervised Hierarchical Multi-Label Text Classification for Product Reviews

DONGYUN NAM, Korea University department of Computer Science, South Korea

Hierarchical Multi-Label Text Classification (HMTc) in unsupervised settings remains a significant challenge due to the lack of ground-truth labels and the complexity of large-scale taxonomies. In this project, I propose a robust classification framework that integrates a DeBERTa-v3-base document encoder with a taxonomy-enhanced Graph Convolutional Network (GCN) label encoder. A pivotal architectural contribution is the implementation of residual Skip Connections within the GCN, which effectively mitigates the over-smoothing problem and improves the Macro-F1 score by 0.4. To generate high-quality training signals, I developed a multi-stage silver label generation pipeline that combines semantic retrieval (BGE-M3 and Cross-Encoder) with selective LLM refinement. Experimental results on the Amazon-531 dataset demonstrate that providing a broad candidate pool to GPT-4o-mini with a strict single-label, hierarchy-aware prompt maximizes classification precision. The final optimized pipeline, further enhanced by iterative self-training, achieved a peak Macro-F1 score of **0.66252**, demonstrating the powerful synergy between structural taxonomic priors and Large Language Model reasoning.

1 Silver Label Generation

1.1 Overview

In the absence of ground-truth labels, primary goal is to generate *reliable* pseudo-labels (silver labels) from unlabeled product reviews while minimizing noise propagation during training. To this end, I design a multi-stage pipeline that progressively refines candidate labels through semantic retrieval, taxonomy-aware confidence scoring, and selective LLM-based refinement. The pipeline consists of four stages: (i) candidate retrieval, (ii) confident core class identification, (iii) LLM refinement for ambiguous cases, and (iv) hierarchical label expansion. This staged design intentionally balances *high recall* in early retrieval with *precision-oriented filtering* in later stages.

1.2 Stage 1: Candidate Retrieval

Two-stage retrieve-then-rerank. Directly matching each document against all classes is prone to semantic ambiguity and hierarchical overlap. Therefore I adopt a retrieve-then-rerank strategy. First, a bi-encoder (BAAI/bge-m3) retrieves a shortlist of semantically relevant candidates. Second, a cross-encoder reranker(BAAI/bge-reranker-v2-m3) computes more accurate relevance scores by jointly encoding each document-class pair. I tried other reranker model, but the bge-reranker-v2-m3 model is powerful for this dataset

Concretely, for each document we retrieve Top- K candidates using cosine similarity over normalized SBERT embeddings, then rerank these candidates with a cross-encoder and convert reranker outputs to probabilities via a sigmoid function. I set $K = 100$ to balance coverage and computational cost: smaller K increases the risk of missing true labels, while larger K introduces more noise and reranking overhead.

Class text representation. Instead of using only the class name, I represent each class by combining (i) its full hierarchical path from the root and (ii) class-specific keywords. This enriched representation reduces ambiguity (e.g., distinguishing similarly named classes under different branches) and improves semantic coverage for retrieval and reranking.

Cross-encoder reranking. While bi-encoders provide fast retrieval, they lack token-level interaction between the document and class description. The cross-encoder processes a paired input, enabling richer matching signals and yielding more reliable relevance probabilities for downstream filtering.

1.3 Stage 2: Confident Core Class Identification

In this stage, I identify a *confident core set* of classes from the initial candidates. A naive approach would be to simply truncate the candidates by taking the top 15 classes based on raw relevance scores. However, this "flat" truncation ignores hierarchical dependencies and introduces significant noise. Therefore, I developed a more robust process based on hierarchical pruning to ensure high-quality silver labels.

Failure of Top-down Search. Initially, I implemented the hierarchical selection method based on the top-down approach, which explores the taxonomy level-by-level using path scores. However, empirical evaluations of this implementation resulted in a poor Macro-F1 score of approximately **0.37**. The rigid nature of the top-down search means that if a parent node is incorrectly discarded due to a low local score, its correct descendants can never be recovered, leading to a fatal loss in recall. To address this, I shifted to a "Flat Retrieval followed by Pruning" strategy, starting with a broad Top-100 candidate pool to ensure high coverage.

Effectiveness of Level-0 Structural Constraints. Through extensive error analysis, I identified that the model's performance degrades significantly when the predicted candidates are scattered across multiple top-level categories. Such dispersion usually indicates "semantic drift" or noisy retrieval, where the model fails to anchor the document within a consistent taxonomic branch. To mitigate this, I applied a **Level-0 structural constraint**: if candidates are distributed across three or more Level-0 roots, I retain only the top two branches based on their maximum raw scores and discard the rest. In a baseline environment without LLM assistance, this heuristic proved to be the most critical factor for performance. Using only raw score filtering ($\tau_{\text{raw}} = 0.5006$) resulted in a Macro-F1 of **0.40715**, but the addition of the **Level-0 constraint** significantly improved the score to **0.44537**, confirming that structural consistency is vital for stabilizing heuristic models.

Interaction between Heuristic Filtering and LLM Reasoning. An interesting phenomenon occurred when integrating the Large Language Model (LLM). While the full heuristic pipeline (including Level-0 constraints) combined with LLM refinement achieved a score of **0.63474**, the best performance of **0.66252** was reached by using *only* the raw score filter before LLM input. This suggests that while the Level-0 constraint is essential for stabilizing weak heuristic models, it becomes too restrictive for an LLM. The LLM's superior reasoning allows it to navigate through a noisier candidate pool and identify correct labels that the heuristic filter would have prematurely pruned.

1.4 Stage 3: LLM Refinement (GPT-4o-mini)

I utilize GPT-4o-mini as a selective refinement module to resolve semantic ambiguities that heuristic filters cannot address. The transition from heuristic-heavy filtering to LLM-centric refinement marked the most significant improvement in overall label reliability.

Input Policy Optimization. The input policy for the LLM was optimized based on the observation that excessive pre-filtering limits the LLM's potential. In the initial version using the Level-0 filter, the LLM was only provided with samples that had exactly one or two remaining Level-0 categories. However, in the final optimized pipeline, this structural filter was removed, and all documents with at least one candidate passing the raw score threshold were passed to the LLM. This expanded input

pool allowed the LLM to process approximately 90% of the data, leveraging its internal knowledge to select the most appropriate classes from a broader and more diverse search space.

Hierarchy-Aware Precision Prompting. To further maximize classification precision, I redesigned the LLM prompt to shift from the initial multi-label output (0–3 classes) to a strict single-selection policy (0–1 class). The candidates are presented with their full hierarchy paths in a "Level 0 > Level 1 > Level 2" format. This structured context provides the LLM with a clear understanding of each category’s specificity, enabling it to accurately identify the most relevant leaf node. The primary rationale for restricting the output to a single core class was to enhance precision and avoid noisy assignments from ambiguous candidates. This optimization led to a substantial performance leap, increasing the Macro-F1 score from 0.6186 to the final peak of **0.66252**.

1.5 Stage 4: Hierarchical Label Expansion

Following the LLM’s selection, I perform **Hierarchical Label Expansion**, where the single selected core class is expanded into a full label set by adding all corresponding ancestors in the taxonomy. This ensures taxonomic consistency, where a leaf-level positive automatically implies the positivity of its parent nodes. During training, I treat descendants of negative nodes as explicit negative samples rather than using zero-masking, as the latter showed no significant performance benefit.

1.6 Reliability Summary

Label reliability is achieved by balancing high-recall retrieval with precise LLM filtering. While structural constraints were vital for the heuristic baseline (0.40715 $\rightarrow$ 0.44537), providing the LLM with a wider candidate pool and a hierarchy-aware single-selection prompt yielded the final peak performance of **0.66252**.

2 Training Process

2.1 Learning Objectives and Data Filtering

The primary objective of the training process is to learn a multi-label text classifier $f(\cdot)$ that maps a document d into a binary label vector $y = [y_1, \dots, y_{|C|}]$. The model is optimized using **Binary Cross Entropy (BCE) with Logits Loss** to handle the multi-label nature of the product reviews. To enhance learning effectiveness, I prioritize data quality through a selective filtering strategy rather than complex masking.

Confidence-based Filtering and Reasoning. The training set is restricted to documents where at least one silver core class exceeds the raw relevance threshold of $\tau_{\text{raw}} = 0.5006$. By excluding documents that do not meet this minimum matching signal, I prevent the model from learning from low-confidence noise during the early stages. This approach ensures that gradient updates are driven by high-reliability silver labels, thereby establishing a stable optimization path and improving the model’s overall learning effectiveness.

2.2 Supervised Warm-up

The training begins with a supervised warm-up phase designed to align document and label representations using high-confidence silver labels. This phase follows a rigorous setup to ensure reproducibility and stability.

Self-Training for Low-Confidence Documents. Following the supervised warm-up, I conducted a targeted self-training phase for documents that initially yielded zero core classes because their scores fell below the 0.5006 threshold. By using the model from the warm-up phase to generate additional silver labels for these previously excluded samples and training for 3 epochs, I successfully

expanded the training coverage. This refinement led to a modest but clear performance improvement, increasing the Macro-F1 score by approximately 0.01.

Training Setup and Expansion. The model is trained for 30 epochs with a learning rate of 5×10^{-5} and a batch size of 64. For each filtered document, positive targets are not limited to the core classes; they are expanded to include all corresponding ancestors in the taxonomy $\mathcal{T} = (\mathcal{C}, \mathcal{R})$ to ensure hierarchical consistency. This structural approach forces the model to learn the inherent relations between parent and sub-classes as defined in the provided hierarchy.

Precision-Oriented Training Logic. In contrast to common zero-masking techniques for uncertain descendants, I treat all descendants of negative nodes as explicit negative samples. This decision is based on the rationale that masking these regions does not significantly improve model performance in this specific task. By treating them as negatives, the model is encouraged to maintain stricter decision boundaries, which directly contributes to higher precision in final label assignments.

3 Model and Prediction Method

3.1 Architecture Overview

The classification framework is built upon a **Dual Encoder** architecture that maps documents and taxonomic labels into a joint 768-dimensional semantic space. This design integrates a Transformer-based document encoder with a Graph Convolutional Network (GCN) as the label encoder, allowing the model to capture both textual semantics and structural dependencies simultaneously.

3.2 Document Encoder

I utilized microsoft/deberta-v3-base as the backbone for text understanding. The document embedding $\mathbf{d} \in \mathbb{R}^{768}$ is extracted from the [CLS] token equivalent and passed through a linear projection layer. This model was selected for its superior ability to capture contextual nuances in product reviews compared to standard BERT-based architectures.

3.3 Label Encoder: GCN with Skip Connections

To explicitly model the hierarchical relationships defined in the taxonomy $\mathcal{T}$, I implemented a **2-layer Graph Convolutional Network (GCN)**. The label embeddings are initialized with DeBERTa-based representations of class names combined with their full hierarchical paths to provide a strong semantic prior. Each GCN layer incorporates a residual connection, where $\mathbf{x}_{out} = \text{Layer}(\mathbf{A}\mathbf{x}_{in}) + \mathbf{x}_{in}$, to preserve the original semantic identity of each class while aggregating structural context from the parent-child adjacency matrix $\mathbf{A}$. Empirical results demonstrated that the addition of these Skip Connections was critical, resulting in a significant increase of approximately **0.04 in F1 score** compared to a vanilla GCN, effectively mitigating the over-smoothing problem inherent in deep graph neural networks.

3.4 Ablation Study: Alternative Approaches

Several alternative architectures were explored but ultimately excluded due to suboptimal performance. While **Hyperbolic GCN** (using the Poincaré Ball) is theoretically suited for representing tree structures, it yielded lower performance on this specific dataset, likely due to optimization complexities. Similarly, adding **preliminary Fully-Connected (FC) layers** before the GCN led to faster overfitting without improving the final classification accuracy. In addition, I evaluated a **Graph Attention Network (GAT)** employing multi-head attention to dynamically weigh taxonomic relationships. While the GAT model achieved a Macro-F1 score nearly identical to that of the GCN with Skip Connections, the GCN-based architecture demonstrated faster convergence in

terms of loss reduction. Due to this superior training efficiency, the GCN with Skip Connections was selected as the final model for the pipeline.

3.5 Prediction and Inference Logic

The final prediction for a document is computed via the dot product between the document embedding $\mathbf{d}$ and the GCN-refined label embeddings $\mathbf{L}$: $\text{Logits} = \mathbf{d} \cdot \mathbf{L}^\top$. The final multi-label output is generated based on specific constraints and a sigmoid-transformed threshold. Candidates with probabilities $P(y|d) > 0.65$ are marked as positive. To comply with the task requirements, each document is assigned at least 2 and at most 3 labels. The threshold number 0.65 is selected. If fewer than two classes pass the 0.65 threshold, the top 2 classes with the highest probabilities are selected regardless of the threshold to satisfy the minimum label requirement. This logic ensures both structural consistency and adherence to the label cardinality constraints. The threshold of 0.65 was experimentally optimized; while performance remained relatively stable across various values, this specific threshold yielded a consistent improvement of approximately 0.02 in F1 score.

4 Experimental Results

4.1 Experimental Setup

The framework was evaluated on the Amazon-531 dataset, utilizing a three-level taxonomy with 531 categories. In this unsupervised HMTc setting, performance is measured by the **Macro-F1 score** obtained from the competition leaderboard. The document representation is generated via microsoft/deberta-v3-base, while the label representations are refined using hierarchy-aware GNN architectures. The absence of ground-truth labels necessitated a multi-stage silver label generation process and iterative refinement.

4.2 Performance Summary

Table 1 outlines the nine distinct stages of experimental optimization. The superscripts correspond to the in-depth technical analysis provided in the following section.

Table 1. Step-wise Macro-F1 performance on the Amazon-531 dataset. Superscripts link to detailed methodology and insights.

Step	Optimization Strategy and Methodology	Macro-F1
1	TaxoClass Baseline Implementation (Top-down Search) ¹	0.37355
2	Transition to Semantic Retrieval (SBERT + Reranker) ²	0.41686
3	Heuristic Anchoring (Level-0 & Ratio Filtering) ³	0.44537
4	Selective LLM Intervention (Ambiguous Level-0=2 Cases) ⁴	0.49407
5	Expanded LLM Refinement (Uncertainty + Ratio Constraints) ⁵	0.57555
6	GNN Architecture Optimization (GCN with Skip Connections) ⁶	0.60316
7	Full Training Corpus Silver Label Generation via LLM ⁷	0.62074
8	Iterative Self-Training (Recovery of LLM-Failed Docs) ⁸	0.62734
9	Final Optimized Strategy (Single-label Policy) ⁹	0.66252

4.3 Analysis of Experimental Stages

Step 1: Evaluation of the Top-down Baseline (0.37355) ⁽¹⁾. The initial stage involved a rigorous implementation of the TaxoClass baseline, including core class mining and supervised warm-up. Despite extensive hyperparameter tuning over 8 epochs, the performance plateaued at 0.37355. While the original paper reported higher scores, the lack of public code suggests that rigid top-down

search acts as a critical bottleneck; classification errors at root levels lead to an irreversible loss of recall for all correct descendants.

Step 2: Backbone Shift to Semantic Retrieval (0.41686) ⁽²⁾. To overcome the search constraints of the baseline, I transitioned to a retrieve-then-rerank backbone using SBERT (BGE-M3) and a Cross-Encoder Reranker. This allowed the model to bypass the rigid hierarchy during initial candidate selection, establishing a more flexible semantic foundation. This shift proved that broad semantic retrieval is more effective than constrained top-down search for large taxonomies.

Step 3: Heuristic Anchoring for Structural Stability (0.44537) ⁽³⁾. I introduced Level-0 structural and count ratio filtering to stabilize reranker outputs. Under the assumption that root-level categories are more semantically distinct and easier to identify, these heuristics served as anchors to suppress semantic drift. This proxy for label quality—identifying consistent Level-0 predictions—directly improved the F1 score to 0.44537.

Step 4: Targeted LLM Intervention (0.49407) ⁽⁴⁾. Selective LLM refinement was introduced for "ambiguous" documents where the heuristic model identified exactly two Level-0 roots. Initially used for methodology verification with a limited epoch count, this stage demonstrated that GPT-4o-mini could resolve high-level conflicts that simple reranking could not, raising the performance to 0.49407.

Step 5: Refinement for High-Uncertainty Documents (0.57555) ⁽⁵⁾. The LLM invocation policy was expanded to include all documents failing the ratio filter or showing structural uncertainty. Targeting LLM resources specifically at the "noisiest" silver labels yielded a substantial performance jump, confirming that LLM reasoning provides the highest marginal utility when applied to high-entropy label candidates.

Step 6: GNN Architectural Optimization (0.60316) ⁽⁶⁾. I explicitly modeled the taxonomic relationships in $\mathcal{T}$ using several GNN architectures to refine label representations. While a vanilla GCN suffered from over-smoothing—making node representations indistinguishable—the implementation of a **GCN with Skip Connections** was significantly more effective. Each layer utilized a residual link, $\mathbf{x}_{out} = \text{Layer}(\mathbf{A}\mathbf{x}_{in}) + \mathbf{x}_{in}$, which preserved the semantic identity of class-specific embeddings while aggregating structural context from the adjacency matrix $\mathbf{A}$. Although a **Graph Attention Network (GAT)** achieved comparable accuracy, the GCN with Skip Connections demonstrated superior training efficiency and faster loss convergence, successfully raising the Macro-F1 score to **0.60316**.

Step 7: Global Silver Label Generation (0.62074) ⁽⁷⁾. The LLM-based refinement was scaled to the entire training corpus. By generating high-precision silver labels for all valid documents, the model's knowledge base became globally consistent. This consistent supervision allowed the DeBERTa encoder to learn clearer decision boundaries across the full dataset.

Step 8: Self-Training for Signal Recovery (0.62734) ⁽⁸⁾. I implemented a 3-epoch iterative self-training phase to process documents where the LLM initially failed to provide a confident prediction. By utilizing the trained model's own predictions to generate pseudo-labels for these low-confidence samples, I recovered previously lost training signals, resulting in a performance gain to 0.62734.

Step 9: Final Optimization and Single-label Policy (0.66252) ⁽⁹⁾. The final peak performance was reached by enforcing a strict "single most specific label" policy in the LLM prompt and removing all heuristic pre-filters. This maximized precision by allowing the LLM to reason over the raw search results without being constrained by intermediate heuristics. This strategy successfully maximized

the synergy between semantic retrieval and LLM reasoning to reach the final leaderboard score of **0.66252**.

4.4 Conclusion of Methodology

Throughout the experiments, performance was optimized through a balance of reranker tuning, heuristic post-processing, and targeted LLM refinement (limited to 1,000 API calls). The integration of GNN structural priors and iterative self-training was essential in refining the decision boundaries for the final classification.

5 Case Study and Discussion

To evaluate the practical performance of the proposed model, I conducted a qualitative analysis of the classification results. This section analyzes specific success and failure cases to provide insights into the model's behavior and limitations by using the best model's prediction. Because I don't know the exact class of the document with certainty, I directly provided the document and candidate classes to Gemini 3.0, manually verified that the returned class label was appropriate, and then proceeded with the case study under the assumption that this verified label was the ground truth for that document.

5.1 Success Analysis

Case 1: Hair Straightening Iron (Document 0). The model correctly predicted the hierarchical label *beauty* → *hair_care* → *styling_tools*. The review for this product contained explicit keywords such as "straightener," "straightening iron," and "thick, long, wavy hair." The success in this case can be attributed to the DeBERTa encoder's ability to capture the functional context of the product. Furthermore, the hierarchy-aware GNN effectively utilized the parent-child relationships, allowing the attention mechanism to reinforce the connection between the specific leaf node (*styling_tools*) and its broader categories.

Case 2: Women's Perfume (Document 10006). In this instance, the model accurately identified the category *beauty* → *fragrance* → *women_s*. The product name itself provided direct clues like "eau de parfum" and "fragrance." Additionally, this case demonstrates the effectiveness of the self-training phase. By reinforcing patterns found in similar perfume reviews during the iterative training process, the model developed a high confidence level for fragrance-related terminology, leading to precise classification.

5.2 Failure Analysis and Model Limitations

Case 3: Ambiguous Label Boundaries (Document 10017). A "Hair Clipper Guard" was misclassified by the model as *beauty* → *hair_care* → *styling_tools*, whereas the ground truth was *health_personal_care* → *personal_care* → *shaving_hair_removal*. This failure highlights the inherent ambiguity in the taxonomy. A clipper guard semantically fits both "styling tools" and "shaving/hair removal." Furthermore, the review was extremely brief (less than 30 words), providing insufficient context for the model to distinguish between these two closely related branches. This suggests that the model struggles when the linguistic difference between distinct taxonomic paths is subtle.

Case 4: Context Bias vs. Metadata (Document 10023). The classification of "Dr. Bronner Castile Soap" presents a notable insight into model behavior. While the ground truth is *beauty* → *bath_body* → *cleansers*, the model predicted *health_personal_care* → *household_supplies* → *household_cleaning*. The review emphasized non-beauty uses such as "washing floors" and "cleaning carpet." Because the DeBERTa encoder relies heavily on the review text, it was biased toward the user's specific

usage pattern rather than the product’s primary market category. This reveals a limitation where the model prioritizes idiosyncratic user context over the intrinsic properties of the product.

5.3 Overall Insights

The analysis confirms that the model is highly effective at identifying products with explicit keyword markers and consistent hierarchical structures. However, two primary limitations remain: **Label Ambiguity** and **Contextual Bias**. Ambiguity arises when a product’s utility spans multiple taxonomic branches, leading to competition between similar label embeddings. Contextual bias occurs when the model over-fits to specific usage descriptions in a review, losing sight of the broader product category. Future improvements could involve integrating product titles or metadata as separate input features to balance the influence of subjective review content.

6 Conclusion

In this project, I developed a taxonomy-aware classification framework integrating DeBERTa-v3-base with a GCN-based label encoder. The implementation of residual **Skip Connections** within the GCN was a pivotal design choice, resulting in a significant **0.4 increase in Macro-F1** by effectively mitigating the over-smoothing problem. While GAT offered comparable accuracy, the GCN-based architecture was selected for its superior convergence speed and training efficiency.

The labeling strategy reached a peak Macro-F1 of **0.66252** through a successful synergy of high-recall retrieval and precision-oriented LLM refinement. While structural heuristics were vital for stabilizing the initial baseline (0.407 $\rightarrow$ 0.445), the final optimal performance was achieved by providing GPT-4o-mini with a broader candidate pool and a **hierarchy-aware single-selection prompt**. Ultimately, this work demonstrates that combining structural taxonomic priors with selective LLM reasoning is a powerful and scalable approach for unsupervised hierarchical classification.